Contract 1261 - Section S320.2.5

Software Module Design Specification

Name	Responsibilities	Date Involved
Kelvin Yuen	Preparation of this document	01/09/2025
Henry Fok	Project oversight and final approval of documentation.	

Revisions

Revision #	Revision Summary	Date
1	First draft	01/09/2025

Revisions 2
1. Introduction 8
1.1 Purpose 8
1.2 Scope 8
1.3 Applicability and audience 8
1.4 Development level and compliance notes 8
2. References 9
2.1 Normative references 9
2.2 Acronyms and glossary 9
2.3 Module-specific terms 9
3. System Overview 10
3.1 System Context 10
3.2 System Architecture 10
3.3 Key Features 10
3.4 Design Constraints 11
3.5 Assumptions and Dependencies 11
4. Module Design 12
4.1 High-Level Architecture 12
4.1.1 File Directory 12
4.2 General Element for plugin of OctoberCMS 12
4.2.1 File Directory 12
4.2.2 Module Components 13
4.3 API Module (dcs/api) 14
4.3.1 File Directory 14
4.3.2 Module Components 14
4.3.3 Module Functions 14
4.3.3.1 API Endpoints 14
4.3.4 Implementation Details 15
4.3.4.1 Headers for request 15
4.3.4.2 Input Parameters for endpoints 15
4.3.4.2.1 api/dcs/oedsdelivery/historical-data 15
4.3.4.2.2 api/dcs/oedsdelivery/historical-data-status 15
4.3.4.3 Checksum algorithm 16
4.3.4.4 Output Parameters for endpoints 16
4.3.4.4.1 api/dcs/oedsdelivery/historical-data 16
4.3.4.4.2 api/dcs/oedsdelivery/historical-data-status 16
4.3.4.5 Additional protection 17
4.3.4.5.1 Protection of check_string 17
4.3.4.5.2 Protection of duplication request 17
4.4 Authentication Manager Module (dcs/authmanager) 17
4.4.1 File Directory 17

4.4.2 Module Components 18
4.4.3 Module Functions 18
4.4.3.1 Overrided the backend sign-in page 18
4.4.3.2 2FA Verification page 18
4.4.4 Implementation Details 19
4.4.4.1 Sign-in function 19
4.4.4.2 Forget Password function 19
4.4.4.3 2FA Verification page 19
4.4.4.4 Sign-out 19
4.5 Common Utilities Module (dcs/common) 19
4.5.1 File Directory 19
4.5.2 Module Components 20
4.5.3 Module Functions 20
4.5.3.1 Provide common functions for extended controllers 20
4.5.3.2 Datetime management 20
4.5.3.3 Random String 20
4.5.3.4 Encryption of string 21
4.5.3.5 Create Email Delivery Request 21
4.5.3.6 Defined constant variables for the plugin 21
4.5.3.7 Support soft deletion of database records 21
4.6 Connection Setting Module (dcs/connsetting) 21
4.6.1 File Directory 21
4.6.2 Module Components 21
4.6.3 Module Functions 22
4.6.3.1 Manages NMS Sftp Setting 22
4.6.3.2 Manages OEDS API Setting 22
4.6.3.3 Manages OEDS Sftp Setting 22
4.7 Dashboard Module (dcs/dashboard) 22
4.7.1 File Directory 22
4.7.2 Module Components 22
4.7.3 Module Functions 23
4.7.3.1 Display the dashboard chart on CMS 23
4.7.3.1.1 Submission Chart 23
4.7.3.1.2 Log Chart 23
4.7.3.1.3 Record Chart 23
4.7.3.1.4 Batch Request Chart 23
4.7.3.2 Provides function to prevent duplicate record for each hour 24
4.7.3.3 Provides four endpoints for the chart to retrieve statistical data 24
4.7.4 Implementation Details 24
4.7.4.1 Display dashboard 24
4.7.4.2 Data request endpoint 24

4.7.4.2.1 subdata_get : Provides data for “Submission Chart” 24
4.7.4.2.2 alarmdata_get : Provides data for “Log Chart” 25
4.7.4.2.3 recorddata_get : Provides data for “Record Chart” 25
4.7.4.2.4 recorddata_get : Provides data for “Batch Request Chart” 25
4.8 Housekeeping Module (dcs/housekeeping) 26
4.8.1 File Directory 26
4.8.2 Module Components 26
4.8.3 Module Functions 26
4.8.3.1 Delete outdated records 26
4.8.3.2 Daily archive record 27
4.8.3.3 Summarize data for dashboard 27
4.9 SFTP file retrieving module (dcs/nmscomm/sftpretrieval) 27
4.9.1 File Directory 27
4.9.2 Module Components 28
4.9.3 Module Functions 28
4.9.3.1 Retrieving log records from NMS SFTP server 28
4.9.4 Implementation Details 28
4.9.4.1 Implementation detail of retrieving logs file from NMS SFTP server 28
4.9.4.2 Implementation detail of parsing the log file, validating the records and inserting log data to database tables 29
4.10 Log Transformation Module (dcs/nmscomm/transformlog) 29
4.10.1 File Directory 29
4.10.2 Module Components 29
4.10.3 Module Functions 30
4.10.3.1 Transforming NMS log records to OEDS requested format 30
4.10.4 Implementation Details 30
4.10.4.1 Implementation detail of transforming NMS log records to OEDS requested format 30
4.10.4.2 Mapping of alarm logs 30
4.10.4.3 Mapping of heartbeat logs 31
4.11 Periodic transfer log records to OEDS through API (dcs/oedscomm/uploadapi) 31
4.11.1 File Directory 31
4.11.2 Module Components 32
4.11.3 Module Functions 32
4.11.4 Implementation Details 32
4.11.4.1 Implementation detail of uploaded periodic log to OEDS 32
4.11.4.2 Retry algorithms 32
4.11.4.3 Logging 32
4.12 Preparing CSV file for historical records request (dcs/oedscomm/csvpreparing) 33
4.12.1 File Directory 33
4.12.2 Module Components 33

4.12.3 Module Functions 33
4.12.4 Implementation Details 33
4.12.4.1 Implementation detail of preparing csv file 33
4.12.4.2 Logging 34
4.13 SFTP upload module for transferring csv file (dcs/oedscomm/uploadsftp) 34
4.13.1 File Directory 34
4.13.2 Module Components 34
4.13.3 Module Functions 35
4.13.4 Implementation Details 35
4.13.4.1 Implementation detail of preparing csv file 35
4.13.4.2 Retry algorithms 35
4.13.4.2 Logging 35
4.14 Verify uploaded file processing status (dcs/oedscomm/verifyupload) 35
4.14.1 File Directory 35
4.14.2 Module Components 36
4.14.3 Module Functions 36
4.14.4 Implementation Details 36
4.14.4.1 Implementation detail of verifying the processed status of files 36
4.14.4.2 Logging 36
4.15 Mail Log Module (dcs/maillog) 36
4.15.1 File Directory 36
4.15.2 Module Components 36
4.15.3 Module Functions 37
4.15.3.1 Record the logs of email delivery 37
4.15.3.2 Delete the outdated logs daily 37
4.15.3.3 Display the Mail Log list 37
4.15.4 Implementation Details 37
4.15.4.1 Status Definition 37
4.15.4.2 Creation the email log records 37
4.15.4.3 Update the email log when email sent successfully 38
4.15.4.4 Update the email log when email failure to send 38
4.15.4.5 Update the email log when queue failed 38
4.16 Logpage Module (dcs/logpage) 38
4.16.1 File Directory 38
4.16.2 Module Components 39
4.16.3 Module Functions 42
4.16.3.1 Show “NMS - Retrieval Log (SFTP)” section 42
4.16.3.2 Show “OEDS - Periodic Upload Log (API)” section 42
4.16.3.3 Show “OEDS - Historical Upload Log (API)” section 43
4.16.3.4 Show “OEDS - Batch Upload Log (SFTP)” section 43
4.16.3.5 Show “OEDS - Batch API Log” record section 43

4.16.4 Implementation Details 44
4.16.4.1 “Retrieved Status” Definition 44
4.16.4.2 “Upload Status” Definition 44
4.16.4.3 Requested type of files 44
4.16.4.4 OEDS Batch upload - File exporting status 45
4.16.4.5 Upload Status 45

1. Introduction

1.1 Purpose

This Software Module Design Specification (SMDS) document provides detailed design information for the DCS system modules. This document describes the individual modules, their functions, interfaces, internal structures, and implementation approaches.

1.2 Scope

The DCS system is built on the OctoberCMS platform and provides API endpoints for OEDS (Operational Engineering Data System) integration. The system handles retrieving logs from NMS, submits logs to OEDS periodically, handles historical data requests and status tracking with secure authentication mechanisms, including data validation, audit logging, and SFTP file handling.

1.3 Applicability and audience

(T.B.D.)

1.4 Development level and compliance notes

(T.B.D.)

2. References

2.1 Normative references

- Contract 1261 - Scope - D/MTRCL/TUE/1261/S/001/A1 - S320.2.5
- Contract 1261 - Scope - D/MTRCL/TUE/1261/S/001/A1 - S320.4
- Contract 1261 - Scope - D/MTRCL/TUE/1261/S/001/A1 - APPENDIX M
- General Specification for Electrical & Mechanical Engineering Works (NEC4) - D/MTRCL/NW/E&M(NEC4)/GS/001/A2 - G13

2.2 Acronyms and glossary

API	Application Programming Interface
CMS	Content Management System
DCS	Data Collection System
OEDS	Operational Engineering Data System
NMS	Network Management System
SFTP	Secure File Transfer Protocol
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP Secure
SQL	Structured Query Language
UUID	Universal Unique Identifier

2.3 Module-specific terms

(T.B.D.)

3. System Overview

3.1 System Context

The DCS System serves as an intermediary between the NMS and the OEDS, facilitating secure historical data requests and status tracking. The system is built on OctoberCMS, leveraging Laravel's robust features for web application development.

3.2 System Architecture

The system follows a modular architecture pattern built on the OctoberCMS platform. The architecture consists of:

- Presentation layer (OctoberCMS backend interface)
- Business logic layer (custom plugins)
- Integration layer (API services)
- Data layer (database and external systems)

3.3 Key Features

The following list is the key modules in this system.

- SFTP file handling for retrieving log from NMS
- Transform log records from NMS log files to corresponding OEDS supported format
- Upload records to OEDS through api
- Provides secure API endpoints with authentication and rate limiting for OEDS retrieving historical data
- Preparing csv file for historical data request
- SFTP file handling for uploading the csv file
- Verify the completeness of uploaded file status through provided OEDS API
- Records for logging of some key status change on most processes
- Comprehensive audit logging
- Configurable system parameters (include SFTP settings)

3.4 Design Constraints

The following list the key design constraints of the system

- Must integrate with existing OctoberCMS infrastructure
- API rate limiting to prevent system overload
- Security requirements for data protection
- Data retention policies for log management

3.5 Assumptions and Dependencies

The following list is the key assumptions and dependencies for the system running normally

- OctoberCMS platform is properly configured and maintained
- External OEDS and NMS systems are accessible
- SFTP server configuration is available
- SSL/TLS certificates are properly configured for secure communication
- SMTP can be accessible to transfer 2FA mail and error alert to administrator

4. Module Design

4.1 High-Level Architecture

The system modules are organized as OctoberCMS plugins with the following structure:

4.1.1 File Directory

```
plugins/
└── dcs/
    ├── api/                # Restful API module
    ├── authmanager/        # Authentication module
    ├── common/             # Common utilities
    ├── connsetting/        # Connection settings
    ├── dashboard/          # Dashboard module
    ├── housekeeping/       # Maintenance utilities
    ├── logpage/            # Logging functionality
    ├── maillog/            # Email logging
    ├── nmscomm/            # NMS communication
    └── oedscomm/           # OEDS communication
```

4.2 General Element for plugin of OctoberCMS

This section describes some general elements for the plugin of OctoberCMS.

4.2.1 File Directory

```
plugin_name/
├── controllers/
        ├── module_name
                ├── _list_toolbar.php
                ├── config_list.yaml
                ├── config_filter.yaml
                ├── config_form.yaml
                ├── config_relation.yaml
                ├── config_import_export.yaml
                ├── create.php
                ├── index.php
                ├── preview.php
                ├── update.php
├── models
        ├── module_name
                ├── columns.yaml
                ├── fields.yaml
├── lang
        ├── en
                ├── lang.php
├── updates/
        ├── create_user_auth_token_table.php
        ├── version.yaml
```

```
├── composer.json
├── plugin.php
├── routes.php
```

4.2.2 Module Components

Folder	File	Definition
controllers/module_name	config_list.yaml	Defines the list information for plugin
	config_filter.yaml	Defines the filter for the list
	config_form.yaml	Defines the edit form information for plugin
	config_relation.yaml	Defines the relation list and form for related table of current records
	config_import_export.yaml	Defines the configuration of import and export function
	_list_toolbar.php	Implements the buttons of toolbars for the list page
	create.php	Defines the layout for create page of record
	index.php	Index page for the module, usually launch the list
	preview.php	Defines the layout for preview page of record
	update.php	Defines the layout for update page of record
models/module_name	columns.yaml	Defines the columns for the list
	fields.yaml	Defines the fields for the update and preview form
lang/en	lang.php	Defines some language variable and description which used in the plugin
updates	xxx.php	Implements the class for create database migrations
	version.yaml	Defines the version control of database migrations
/	plugin.php	Describe the plugin detail and define the middleware class
	routes.php	Define the request path to specific view or function
	composer.json	Json for installing the package by composer

4.3 API Module (dcs/api)

This module is used to implement the API interface for the OEDS system to request historical records and check the preceding status of the request.

4.3.1 File Directory

```
api/
├── classes/
        ├── HistoricalDataService.php
├── controllers/
        ├── HistoricalData.php
├── middleware/
        ├── ApiKeyMiddleware.php
├── settings/
        ├── ApiConstant.php
```

4.3.2 Module Components

Folder	File	Definition
classes	HistoricalDataService.php	Function serves for API requests - Security validation (checksum, field validation) - Logging
controllers	HistoricalData.php	Function handling the code for two apis - Request historical data - Check the status of historical data request
middleware	ApiKeyMiddleware.php	Validation of X-API-Key of api request
settings	ApiConstant.php	Define the constants for API classes

4.3.3 Module Functions

4.3.3.1 API Endpoints

Path	Method	Function
api/dcs/oedsdelivery/historical-data	Post	For OEDS to request retrieving historical logs data
api/dcs/oedsdelivery/historical-data-status	Post	For OEDS to check the status of retrieving request

4.3.4 Implementation Details

4.3.4.1 Headers for request

Variable	Value	Description
Content-Type	application/json	Indicates the request format is in json
Accept	application/json	Indicates accepting json as the return format
X-API-Key	<Your-API-Key>	A security key to protect the api requested by authorized party

4.3.4.2 Input Parameters for endpoints

4.3.4.2.1 api/dcs/oedsdelivery/historical-data

Variable	Value	Description
start_date	string	The start date (in YYYY-MM-DD) of the records to be exported.
end_date	string	The end date (in YYYY-MM-DD) of the records to be exported. Must be equal and later than start date
request_type	json	Type of requested data in json format ([“1”] or ["1", "2"]). 1 = Heart beat 2 = Alarm
check_string	string	A random string of 16 chars [a-zA-Z0-9]
cc	string	Checksum Md5(“check_string” + “start_date” + “end_date” + “request_type”) e.g. Md5(wjeuhrn3hew32hej2025-10-022025-10-03["1","2"])

4.3.4.2.2 api/dcs/oedsdelivery/historical-data-status

Variable	Value	Description
request_id	int	Returned id in request 4.2.4.2.1
check_string	string	A random string of 16 chars [a-zA-Z0-9]
cc	string	Checksum Md5(“check_string” + “request_id”) e.g. Md5(wjeuhrn3hew32hej123)

4.3.4.3 Checksum algorithm

This is additional protection to protect api from being called by unauthorized parties. MD5 of undisclosed patterns of parameters are applied to archive the goal.

4.3.4.4 Output Parameters for endpoints

4.3.4.4.1 api/dcs/oedsdelivery/historical-data

Response HTTP Code	Sample Response

200	{
	"message": "Request received.",
	"id": 123
	}

400	{
	"errors": [{
	"name": "start_date",
	"message": "Start date is required and cannot be empty"
	},
	{
	"name": "end_date",
	"message": "End date must be after start date"
	}] }

4.3.4.4.2 api/dcs/oedsdelivery/historical-data-status

Response HTTP Code	Sample Response

200	{
	"status": 1,
	"status_desc": "Request success, file already upload.",
	"file_name": “DCS_ONDEMAND_20251002000000_123.zip”
	}

400	{
	"error": "Unauthorized"
	}

4.3.4.5 Additional protection

4.3.4.5.1 Protection of check_string

- The same check_string cannot re-use in 1 month. For the same request with the same check_string is count as retry request

Response HTTP Code	Sample Response

400	{
	"errors": [{
	"name": "check_string",
	"message": "check_string already in use"
	}]
	}

4.3.4.5.2 Protection of duplication request

- Cannot have same start_date, end_date, request_type in 1 hour (Same request already exist)

Response HTTP Code	Sample Response

400	{
	"errors": [{
	"name": "start_date",
	"message": "A similar request has been made in the last hour.”
	}]
	}

4.4 Authentication Manager Module (dcs/authmanager)

This module is used to extend the platform backend portal authentication and implement the 2FA authentication supported with Email.

4.4.1 File Directory

```
authmanager/
├── controllers/
        ├── auth
                ├── signin.htm
                ├── verifysignin.htm
        ├── Auth.php
├── models/
        ├── Auth.php
        ├── AuthToken.php
```

```
├── views/
        ├── mail
                ├── twofamail.htm
```

4.4.2 Module Components

Folder	File	Definition
controllers/auth	signin.htm	Html for backend signin page
	verifysignin.htm	Html for backend 2FA verify page
controllers	Auth.php	Handles the function for Backend Auth. It includes customized a new signin page, verify signin, handles 2FA notification and verification
models	Auth.php	Works with database table dcs_auths
	AuthToken.php	Works with database table dcs_cmsuser_auth_token and implements the creation and verification function for 2FA token
views/mail	twofamail.htm	Defines the email template of 2FA token notification

4.4.3 Module Functions

4.4.3.1 Overrided the backend sign-in page

The default CMS sign-in page is overridden by this plugin. After verification of username and password, a random code will be generated and delivered to the user mailbox. CMS page will be redirected to the 2FA verification page.

Support “Forget Password” on the sign-in page. Enter username on “Forget Password” page and then an email with reset password link will be delivered to the user mailbox. Users can open the link to enter a new password. Once password reset, users can sign-in with new password

4.4.3.2 2FA Verification page

Supports entering received 2FA code, CMS will redirect to Dashboard page once verified successfully. If an email is not received, the user can click “Resend Token” after passing one minute of cooling period. A new email will be delivered to user mail with a new password.

4.4.4 Implementation Details

4.4.4.1 Sign-in function

The Sign-in page is controlled by /controllers/Auth.php::sign-in, it will load the auth/signin.htm page and show it to the user. Users need to enter username and password and then click the Submit button. The browser will post the request to this “sign-in” function again to do authentication (/controllers/Auth.php::authenticate + /controllers/Auth.php::validateInternal). Once authenticated, it will generate a random number in 6 digits (/controllers/Auth.php::generate_token), put it in the “mail/twofamail.htm” template and then deliver it to the user mailbox.

4.4.4.2 Forget Password function

This function is implemented by the default OctoberCMS function.

4.4.4.3 2FA Verification page

The 2FA Verification page is controlled by /controllers/Auth.php::verifysignin, it will load the auth/verifysignin.htm page and show it to the user. Users need to enter the received token and then click the Submit button. The browser will post the request to this “verifysignin” function again to do verification (/controllers/Auth.php::handleVerifyToken), if successful, will redirect to the CMS Dashboard page.

When a user clicks “Resend Token”, the browser will post to /controllers/Auth.php::resenttoken and a new token will be delivered to the user mailbox. “Resend Token” button will be counted down by the 60 seconds and will be updated in each second. The displayed prefix will be updated too to prevent messing with expired tokens.

4.4.4.4 Sign-out

The sign-out is controlled by /controllers/Auth.php::sign-out function. It will redirect to sign-in page again once successful.

4.5 Common Utilities Module (dcs/common)

This module is used to provide some common utilities for programme development.

4.5.1 File Directory

```
common/
├── assets/
        ├── css
                ├── common.css
        ├── js
                ├── chart.js
├── controllers/
        ├── CommonController.php
```

```
├── models
        ├── CommonFunction.php
        ├── EmailFunction.php
├── setting
        ├── CommonConstant.php
├── traits
        ├── LegatoSoftDelete.php
```

4.5.2 Module Components

Folder	File	Definition
assets/css	common.css	Implements the css which can be loaded for the controller. Currently, it uses to adjust the fontsize of left panel in backend
assets/js	chart.js	Contains the library for displaying the charts in dashboard view
controllers	CommonController.php	Implements some common function for the controller classes. Extend to this class before using the function
models	CommonFunction.php	Implements some common functions for both controllers and models. The functions can be called after creating the object of this class. Currently, it used to handle datetime, random string and encryption of string
	EmailFunction.php	Implements sending email related functions. Loads the template and add the mail to queue
setting	CommonConstant.php	Defines some common variable and can be used by other classes
traits	LegatoSoftDelete.php	This traits for implementing soft delete for database records

4.5.3 Module Functions

4.5.3.1 Provide common functions for extended controllers

controllers/CommonController.php allows controllers to extend it. It contains some common functions for extended classes to be used. For example, getRecordId from url link

4.5.3.2 Datetime management

Functions to manager datetime. It includes “Get UTC Time”, “Get current datetime object in different timezone”, “Get datetime object from string”, “Add / Minus time from datetime object”, “Validate DateTime”. All those functions provided by models/CommonFunction.php

4.5.3.3 Random String

Function to get random strings in different formats and lengths. This function is provided by models/CommonFunction.php

4.5.3.4 Encryption of string

Functions to encrypt and decrypt strings in AES256 format. These functions are provided by models/CommonFunction.php

4.5.3.5 Create Email Delivery Request

Functions to prepare and create email requests. And then adding to the Email delivery queue. Then, OctoberCMS default email module will help to deliver the mail. These functions are provided by models/EmailFunction.php

4.5.3.6 Defined constant variables for the plugin

Define the constant variables for other controllers and models to use.

4.5.3.7 Support soft deletion of database records

Traits to provide soft deletion of database records. It can be used by any models to undergo soft deletion of records. This is provided by traits/LegatoSoftDelete.php

4.6 Connection Setting Module (dcs/connsetting)

This module is used to manage the setting of SFTP authentication to both NMS and OEDS systems.

4.6.1 File Directory

```
connsetting/
├── controllers/
        ├── NmsSftp.php
        ├── OedsApi.php
        ├── OedsSftp.php
├── models
        ├── NmsSftp.php
        ├── OedsApi.php
        ├── OedsSftp.php
```

4.6.2 Module Components

Folder	File	Definition
controllers	NmsSftp.php	Handles the update form of CMS “Connection Setting -> NMS Sftp Setting” page
	OedsApi.php	Handles the update form of CMS “Connection Setting -> OEDS API Setting” page
	OedsSftp.php	Handles the update form of CMS “Connection Setting -> OEDS Sftp Setting” page
models	NmsSftp.php	Works with dcs_connsetting_nmssftp table and set rules for the fields

	OedsApi.php	Works with dcs_connsetting_oedsapi table and set rules for the fields
	OedsSftp.php	Works with dcs_connsetting_oedssftp table and set rules for the fields

4.6.3 Module Functions

4.6.3.1 Manages NMS Sftp Setting

Provides CMS interface (“Connection Setting -> NMS Sftp Setting ”) to manage the connection , authentication, frequency, Prefix of logs and storage location setting for retrieving log files from NMS’s Sftp Server.

4.6.3.2 Manages OEDS API Setting

Provides CMS interface (“Connection Setting -> OEDS API Setting ”) to manage the url and user account information of OEDS API.

4.6.3.3 Manages OEDS Sftp Setting

Provides CMS interface (“Connection Setting -> OEDS SFTP Setting ”) to manage the connection, authentication to OEDS Sftp Server. In addition, the filename and maximum allowed filesize can be adjusted. Finally, the file status verification URL can be changed on this page too.

4.7 Dashboard Module (dcs/dashboard)

This module is used to show the dashboard on CMS and prepare the data for those dashboard views.

4.7.1 File Directory

```
dashboard/
├── controllers/
        ├── dashboardview
                ├── _dashboardview.htm
        ├── DashboardView.php
├── models
        ├── DashboardView.php
```

4.7.2 Module Components

Folder	File	Definition
controllers/das	_dashboardview.htm	Implements the function of the “CMS - Dashboard” page. It

hboardview		currently shows four summary charts and handles the update of graphs after a range of date changes.
controllers	DashboardView.php	Controls the page of “CMS - Dashboard”. Prepares the api function to retrieve the summary data once the range of date changes.
models	DashboardView.php	Works with dcs_dashboard_dashboardview table. Provides functions for obtaining the dataset for different charts in different date range

4.7.3 Module Functions

4.7.3.1 Display the dashboard chart on CMS

4.7.3.1.1 Submission Chart

Shows the NMS retrieval and OEDS periodic submission statistics in a chart. It includes the following four summary indexes in the date. Allows to select a different range of date but maximum interval of date is 30.

1. Number of NMS log files which are processed successfully
2. Number of retried attempts to retrieve the NMS log file
3. Number of OEDS successfully periodic submission (API)
4. Number of retried attempts of OEDS periodic submission (API)

4.7.3.1.2 Log Chart

Shows the number of log records retrieved for each different type of log in a chart. Allows to select a different range of date but maximum interval of date is 30.

1. Number of Alarm log records received in the date
2. Number of Heartbeat log records received in the date

4.7.3.1.3 Record Chart

Shows the statistics for the processed log records in a chart. Allows to select a different range of date but maximum interval of date is 30.

1. Total number of retrieved log records from NMS server in the date
2. Total number of submitted log records to OEDS server in the date

4.7.3.1.4 Batch Request Chart

Shows the statistics for the Batch Request in a chart. Allows to select a different range of date but maximum interval of date is 30.

1. Total number of batch request received
2. Total number of submitted files for received batch request

3. Total number of submitted records for received batch request

4.7.3.2 Provides function to prevent duplicate record for each hour

Provides add_record function to prevent adding duplicate records for each hour in dcs_dashboard_dashboardview table. It will also verify proper datetime format and all statistical data must be integer.

4.7.3.3 Provides four endpoints for the chart to retrieve statistical data

Provides four endpoints for charts to retrieve statistical data for different charts in requested date range. It will check the interval of date range, return error if exceeded the limitation.

4.7.4 Implementation Details

4.7.4.1 Display dashboard

When the CMS dashboard page is loaded, the controller will load “dashboardview.htm” and show the content on the browser. Then, jquery functions will initialize requests to four data endpoints (dashboardView/xxx) to retrieve the data of charts. Chart data will be updated when the endpoint returns data successfully, do nothing when the endpoint fails to return data.

4.7.4.2 Data request endpoint

Following four endpoints (dashboardView/xxx) are implemented to return data to chart

4.7.4.2.1 subdata_get : Provides data for “Submission Chart”

Kind	Parameter	Explanation
Input parameter	sdate	Start Date (in YYYY-MM-DD format)
	edate	End Date (in YYYY-MM-DD format)
Output parameter	result	1 => Success, 0 => Failed
	emsg	Only returned when failed, explanation of the failed reason
	sub_labels	Return array of date labels (x-asix)
	sub_data[‘nms_files’]	Return array of “Number of NMS log files which are processed successfully”
	sub_data[‘nms_retires’]	Return array of “Number of retried attempts to retrieve the NMS log file”
	sub_data[‘oeds_sub’]	Return array of “Number of OEDS successfully periodic submission (API)”

	sub_data[‘oeds_sub_retries’]	Return array of “Number of retried attempts of OEDS periodic submission (API)”

4.7.4.2.2 alarmdata_get : Provides data for “Log Chart”

Kind	Parameter	Explanation
Input parameter	sdate	Start Date (in YYYY-MM-DD format)
	edate	End Date (in YYYY-MM-DD format)
Output parameter	result	1 => Success, 0 => Failed
	emsg	Only returned when failed, explanation of the failed reason
	alarm_labels	Return array of date labels (x-asix)
	alarm_data[‘alarm’]	Return array of “Number of Alarms Log records retrieved”
	alarm_data[‘heartbeat’]	Return array of “Number of Heartbeat Log records retrieved”

4.7.4.2.3 recorddata_get : Provides data for “Record Chart”

Kind	Parameter	Explanation
Input parameter	sdate	Start Date (in YYYY-MM-DD format)
	edate	End Date (in YYYY-MM-DD format)
Output parameter	result	1 => Success, 0 => Failed
	emsg	Only returned when failed, explanation of the failed reason
	record_labels	Return array of date labels (x-asix)
	record_data[‘nms_record’]	Return array of “Total number of retrieved log records from NMS server in the date”
	record_data[‘oeds_sub_record’]	Return array of “Total number of submitted log records to OEDS server in the date”

4.7.4.2.4 recorddata_get : Provides data for “Batch Request Chart”

Kind	Parameter	Explanation
Input parameter	sdate	Start Date (in YYYY-MM-DD format)
	edate	End Date (in YYYY-MM-DD format)

Output parameter	result	1 => Success, 0 => Failed
	emsg	Only returned when failed, explanation of the failed reason
	batchreq_labels	Return array of date labels (x-asix)
	batchreq_data[‘batch_api_received’]	Return array of “Total number of batch request received”
	batchreq_data[‘batch_api_sub’]	Return array of “Total number of submitted files for received batch request”
	batchreq_data[‘batch_api_sub_record’]	Return array of “Total number of submitted records for received batch request”

4.8 Housekeeping Module (dcs/housekeeping)

This module is used to manage the housekeeping of logs data. Delete the outdated logs, archive some live records daily and prepare summary for dashboard daily.

4.8.1 File Directory

```
housekeeping/
├── controllers/
        ├── Housekeeping.php
├── models
        ├── Housekeeping.php
```

4.8.2 Module Components

Folder	File	Definition
controllers	Housekeeping.php	Implements the entry point for gathering summary data and generating demo data
models	Housekeeping.php	Provides functions for deleting some outdated records daily, archiving records daily, summarizing dashboard records hourly and storing the result to dcs_dashboard_dashboardview table.

4.8.3 Module Functions

4.8.3.1 Delete outdated records

This operation (models/Housekeeping::deleteoutdatrecord) is scheduled daily at 00:05.

Records of NMS retrieval, OEDS periodic and batch upload submission action will be deleted after five years since creation.

Log data retrieved will be deleted after five years since creation.

Transformed Log data will be deleted after five years since creation.

Audit log of NMS retrieval, Audit log of OEDS periodic submission, Log of OEDS batch request related api, API logs and system audit logs will be deleted after two years since creation.

4.8.3.2 Daily archive record

This operation (models/Housekeeping::dailyarchiverecord) is scheduled daily at 00:10.

Archive OEDS periodic update data and record to archive tables after two days since creation.

4.8.3.3 Summarize data for dashboard

This operation (models/Housekeeping::summary_to_dashboard) is scheduled hourly at xx:03.

Summarises last hour data and stores them in the dashboard table.

4.9 SFTP file retrieving module (dcs/nmscomm/sftpretrieval)

This module is used to retrieve the logs file from the NMS server. Parsing the records from files and storing them in the database. The file will then move to a local folder for archiving. The authentication information is stored in a database and manageable in CMS.

4.9.1 File Directory

```
nmscomm/
├── controllers/
        ├── SftpRetrieval.php
├── models/
        ├── traits/
                ├── CsvValidatable.php
                ├── SftpDownloadable.php
        ├── RawActiveAlarmLog.php
        ├── RawAlarmLog.php
        ├── RawEquipmentSettingsLog.php
        ├── RawHeartbeatLog.php
        ├── RawInformationLog.php
        ├── RawSoftwareVersionsLog.php
        ├── RawSystemLog.php
        ├── RawUnacknowledgedAlarmsLog.php
        ├── RawUserSettingsLog.php
```

4.9.2 Module Components

Folder	File	Definition
controllers	SftpRetrieval.php	Implements the tasks to control downloading log files from NMS Sftp server
models/traits	CsvValidatable.php	Handles the validation on csv header and data
	SftpDownloadable.php	Provides the function to download the file from Sftp server
models	RawActiveAlarmLog.php	Works with dcs_raw_activealarm_log table.
	RawAlarmLog.php	Works with dcs_raw_alarm_log table.
	RawEquipmentSettingsLog.php	Works with dcs_raw_equipmentsettings_log table.
	RawHeartbeatLog.php	Works with dcs_raw_heartbeat_log table.
	RawInformationLog.php	Works with dcs_raw_information_log table.
	RawSoftwareVersionsLog.php	Works with dcs_raw_softwareversions_log table.
	RawSystemLog.php	Works with dcs_raw_system_log table.
	RawUnacknowledgedAlarmsLog.php	Works with dcs_raw_unacknowledgedalarms_log table.
	RawUserSettingsLog.php	Works with dcs_raw_usersettings_log table.

4.9.3 Module Functions

4.9.3.1 Retrieving log records from NMS SFTP server

Connecting to the NMS SFTP server, retrieving those useful modified files since last retrieval to a local folder. The module will load the csv files, validate the log records and insert log data to “raw_” database tables.

4.9.4 Implementation Details

4.9.4.1 Implementation detail of retrieving logs file from NMS SFTP server

Firstly, the download task (SftpRetrieval -> executeDownloadTask) will be triggered for every x minutes (x is changeable on CMS - refer 4.6.3.1 -> “File Pulling Frequency (minute)”). It will

scan all the files from the SFTP folder, only files with matched prefixes (changeable on CMS - refer 4.6.3.1 -> “Alarm Logs (Prefix)” & “Heartbeat (Prefix)”) and modified since last retrieval time will be processed. it will add a record to the audit log (NmsAuditlog) with status = PULLED.

4.9.4.2 Implementation detail of parsing the log file, validating the records and inserting log data to database tables

Another background thread (CSVParsing -> executeParsingTask) is implemented to parse the pulled log file. This thread will start every minute and keep looping with 0.2s sleep. It then stops at every minute’s 57s. It will look for “PULLED” audit log record, load the corresponding file and update the audit log record to status = “START PARSING”. Parsing and validating the record data. If successful, insert the data to the corresponding “raw_xxx” table (status = ACTIVE) and update the audit log record to status = “PARSING SUCCESSFULLY”. If failed, will trigger the download thread to retry downloading again and update the audit log record to status = “PARSING FAILED”.

4.10 Log Transformation Module (dcs/nmscomm/transformlog)

This module is used to transform the stored log records to OEDS requested format. Transformed records will then be saved back to the database and waiting for transferring to OEDS.

4.10.1 File Directory

```
nmscomm/
├── controllers/
        ├── TransformLog.php
├── models/
        ├── TfdActiveAlarmLog.php
        ├── TfdAlarmLog.php
        ├── TfdEquipmentSettingsLog.php
        ├── TfdHeartbeatLog.php
        ├── TfdInformationLog.php
        ├── TfdSoftwareVersionsLog.php
        ├── TfdSystemLog.php
        ├── TfdUnacknowledgedAlarmsLog.php
        ├── TfdUserSettingsLog.php
```

4.10.2 Module Components

Folder	File	Definition
controllers	TransformLog.php	Implements the tasks to transform log records to required format of OEDS system
models	TfdActiveAlarmLog.php	Works with dcs_tfd_activealarm_log table.

	TfdAlarmLog.php	Works with dcs_tfd_alarm_log table.
	TfdEquipmentSettingsLog.php	Works with dcs_tfd_equipmentsettings_log table.
	TfdHeartbeatLog.php	Works with dcs_tfd_heartbeat_log table.
	TfdInformationLog.php	Works with dcs_tfd_information_log table.
	TfdSoftwareVersionsLog.php	Works with dcs_tfd_softwareversions_log table.
	TfdSystemLog.php	Works with dcs_tfd_system_log table.
	TfdUnacknowledgedAlarmsLog.php	Works with dcs_tfd_unacknowledgedalarms_log table.
	TfdUserSettingsLog.php	Works with dcs_tfd_usersettings_log table.

4.10.3 Module Functions

4.10.3.1 Transforming NMS log records to OEDS requested format

Transforming the received NMS log records to OEDS requested format, storing it in the database and waiting for transfer in next periodic submission.

4.10.4 Implementation Details

4.10.4.1 Implementation detail of transforming NMS log records to OEDS requested format

Background thread (TransformLog -> executeTransformTask) is implemented to transform NMS log records to OEDS requested form. This thread will start every minute and keep looping with 0.2s sleep. It then stops at every minute's 57s. It looks for "ACTIVE" records in raw tables, before transforming, updating the record status to "PENDING". Then, transform the record to OEDS requested format (mapping refers to 4.10.4.2 and 4.10.4.3) and then insert the transformed records into corresponding "tfd_" tables. Finally, update the record status to "TRANSFORMED".

4.10.4.2 Mapping of alarm logs

Parameters	Parameter array	Mapped value (raw->xxx)
upload_json array	Timestamp	event_dt
	EquipmentID	equipment_id

	PointID	userfield_1
	MessageType	severity
	Value	N/A
	Remark	message
log_dt	-	event_dt

4.10.4.3 Mapping of heartbeat logs

Parameters	Parameter array	Mapped value (raw->xxx)
upload_json array	Timestamp	event_dt
	EquipmentID	equipment_id
	PointID	userfield_1
	MessageType	N/A
	Value	N/A
	Remark	message
log_dt	-	event_dt

4.11 Periodic transfer log records to OEDS through API (dcs/oedscomm/uploadapi)

This module is used to transfer new received log records to OEDS through the provided API.

4.11.1 File Directory

```
oedscomm/
├── controllers/
        ├── UploadApi.php
├── helpers
        ├── ApiUploadLogHelper.php
        ├── AuditLogHelper.php
├── models/
        ├── UploadApiFunction.php
├── views/
        ├── mail/
                ├── periodic_push_failed.htm
```

4.11.2 Module Components

Folder	File	Definition
controllers	UploadApi.php	Implements the tasks to handle periodic upload log records to OEDS system
helpers	ApiUploadLogHelper.php	Manages the “OEDS API Upload Logs” during the periodic upload function
	AuditLogHelper.php	Manages the “OEDS Audit Logs” for both periodic upload and historical sftp upload process
models	UploadApiFunction.php	Provides function to upload transformed log record to OEDS server
views/mail	periodic_push_failed.php	Contains email template form sending warning to administrator when period push failed

4.11.3 Module Functions

Periodic uploaded log records to OEDS system through provided API endpoint. The interval is one minute

4.11.4 Implementation Details

4.11.4.1 Implementation detail of uploaded periodic log to OEDS

Every minute, a new thread (UploadApi -> executePeriodicPush) will be called to upload log records to the OEDS system through the provided API endpoint. It will get the “ACTIVE” records from “tfd_” tables and then upload them to OEDS API following the required API specification. It includes generating JWT and posting the data to the API endpoint.

4.11.4.2 Retry algorithms

When the API upload call is fail we will update the table dcs_logpage_oedsapiuploadloglive next_retry_dt and upload_status to waiting_retry, the first time upload will be retry immediate, after that if fail again then will update the "next_retry_dt" to current time +60 sec as the next retry time. Only retry when the respond status on 429 / 5xx / timeout. The max attempts is 5 times.

4.11.4.3 Logging

Every minute, if there are pending “tfd_” records, a summary record will be created and stored on dcs_logpage_oedsapiuploadloglive and dcs_logpage_oedsapiuploadloglive_data tables.

The audit logs of the uploading process will be stored on dcs_oeds_audit_log tables. It includes following action :

- Before posting to API
- Post successfully
- Post failed

4.12 Preparing CSV file for historical records request (dcs/oedscomm/csvpreparing)

This module is used to handle historical records retrieval requests. Retrieving transformed records from database and storing into a csv file. Update the status of historical records request.

4.12.1 File Directory

```
oedscomm/
├── controllers/
        ├── CsvPreparing.php
├── models/
        ├── CsvPreparingFunction.php
```

4.12.2 Module Components

Folder	File	Definition
controllers	CsvPreparing.php	Handles the CSV preparing task for historical upload tasks
models	CsvPreparingFunction.php	Provide functions for obtaining data from database and stored in exporting csv file

4.12.3 Module Functions

Preparing historical records for requested date range, storing them on a CSV file and then waiting for the upload module to transfer.

4.12.4 Implementation Details

4.12.4.1 Implementation detail of preparing csv file

Every minute, a new thread (CsvPreparing -> executeTask) will be called.

- Firstly, it will get all records (dcs_logpage_oedsapiuploadloglive) whose upload_status = “History Received” from dcs_logpage_oedssftpuploadlog table. When it starts processing a record, its upload_status will be changed to “File Start Preparing”.
- Then, it will try to get the log records from “tfd_” tables. If records existed, storing them in a csv file and waiting for upload.
- Once the file is prepared, its upload_status will be changed to “File Prepared Successfully”.

- If no records are found, its upload_status will be changed to “Upload End Successfully" + number of records = 0.
- If any exception is thrown on the process and not yet exceeded the limit of retry attempt, its upload_status will be changed to “History Received” again and wait for retry. “noofattempt” will be added by 1
- If any exception is thrown on the process and exceeds the limit of retry attempt, its upload_status will be changed to “File Prepared Failed”.

4.12.4.2 Logging

The audit logs of the uploading process will be stored on dcs_oeds_audit_log tables. It includes following action :

- Start preparing file process
- File prepared successfully
- File prepared failure

4.13 SFTP upload module for transferring csv file (dcs/oedscomm/uploadsftp)

This module is used to upload the csv file to OEDS SFTP server. The authentication information is stored in a database and manageable in CMS.

4.13.1 File Directory

```
oedscomm/
├── classes/
        ├── SftpService.php
        ├── OedsApiService.php
├── controllers/
        ├── UploadSftp.php
├── models/
        ├── UploadSftpFunction.php
```

4.13.2 Module Components

Folder	File	Definition
classes	SftpService.php	Provides functions to upload file to OEDS Sftp server
	OedsApiService.php	Handles verification of files uploaded status by calling OEDS Files status api
controllers	UploadSftp.php	Manages the uploading files to OEDS SFTP task
models	UploadSftpFunction.php	Provides functions to manage the upload process and the log

4.13.3 Module Functions

Upload generated historical csv file to OEDS SFTP service. Then call provided “List processed files” to verify whether file has been processed by OEDS.

4.13.4 Implementation Details

4.13.4.1 Implementation detail of preparing csv file

Every minute, a new thread (UploadSftp -> executeTask) will be called.

- Firstly, it will get all records (dcs_logpage_oedsapiuploadloglive) whose upload_status = “File Prepared Successfully” from dcs_logpage_oedssftpuploadlog table. When it starts processing a record, its upload_status will be changed to “Upload start” and “noofattempt” will be reset to 0
- Once the file is uploaded completed. Its upload_status will be changed to “Upload end successfully”
- If upload failed and not yet exceeded the limit of retry attempt, its upload_status will be changed to “File Prepared Successfully” again and wait for retry. “noofattempt” will be added by 1
- If upload failed and exceeded the limit of retry attempt, its upload_status will be changed to “Upload end failed”

4.13.4.2 Retry algorithms

(T.B.D.)

4.13.4.2 Logging

The audit logs of the uploading process will be stored on dcs_oeds_audit_log tables. It includes following action :

- Upload start
- Upload start (Retry)
- Upload end successfully
- Upload end failed

4.14 Verify uploaded file processing status (dcs/oedscomm/verifyupload)

This module is used to verify the processed status of files which are uploaded to the OEDS SFTP server.

4.14.1 File Directory

```
oedscomm/
├── classes/
├── controllers/
```

```
├── models/
```

4.14.2 Module Components

4.14.3 Module Functions

4.14.4 Implementation Details

4.14.4.1 Implementation detail of verifying the processed status of files

4.14.4.2 Logging

The audit logs of the uploading process will be stored on dcs_oeds_audit_log tables. It includes following action :

- Verify files successfully

4.15 Mail Log Module (dcs/maillog)

This module is used to store the email log and the administrator can review the mail list in the CMS settings page.

4.15.1 File Directory

```
maillog/
├── console/
        ├── Purge.php
├── controllers
        ├── maillogs/
                ├── _attachments.htm
        ├── MailLogs.php
├── models
        ├── MailLogs.php
        ├── Settings.php
```

4.15.2 Module Components

Folder	File	Definition
console	Purge.php	Provides function to delete outdated mail logs daily
controllers/maillogs	_attachments.htm	Shows the attachments list in mail record

controllers	MailLogs.php	Controls the page of “CMS - Setting - Mail Log”.
models	MailLogs.php	Works with maillog_log table and provides function for control fields in maillog_log table
	Settings.php	Controls the edit form of “CMS - Setting - Mail Log Settings”

4.15.3 Module Functions

4.15.3.1 Record the logs of email delivery

Records the logs of all email delivered in the platform. It includes logging the successful, failure and exception status.

4.15.3.2 Delete the outdated logs daily

Delete the logs whose creation days > 730 days (This value is configurable on CMS - Settings -> Mail Log Settings page). The field is “Keep logs for x Days”.

4.15.3.3 Display the Mail Log list

The maillog list will be displayed to the CMS Administrator. The page is located on Settings -> “Mail Log”.

4.15.4 Implementation Details

4.15.4.1 Status Definition

The status of the record shows the result of the mail record

Value	Meaning
2	Pending record. This email is starting to send
5	Success record. This email is sent successfully
6	Failure record. This email fails to be sent and Error Message will be recorded.

4.15.4.2 Creation the email log records

On boot function of the plugin, the system will trigger createFromMailerSendEvent function to create a log record in “maillog_log” table with “PENDING” status when “mailer.prepareSend” event is received.

4.15.4.3 Update the email log when email sent successfully

On boot function of the plugin, the system will trigger updateMailerToComplete function to update the log record with the same subject, template and pending status in the last 12 hours to “SENT” status when “mailer.send” event is received.

4.15.4.4 Update the email log when email failure to send

On boot function of the plugin, the system will trigger updateMailerToFailed function to update the log record with the same subject, template and pending status in the last 12 hours to “ERROR” status when “mailer.senderror” event is received.

4.15.4.5 Update the email log when queue failed

On boot function of the plugin, the system will trigger updateMailerToFailed_exception function to update the log record with the same template and pending status in the last 12 hours to “ERROR” status when the queue is failing an event whose name is “October\Rain\Mail\Mailable” .

4.16 Logpage Module (dcs/logpage)

This module is used to store logs for the features. It includes the NMS file retrieval logs, both OEDS periodic upload and batch request logs.

4.16.1 File Directory

```
logpage/
├── controllers
        ├── nmssftpretrievallog/
                ├── _completed_dt.htm
                ├── _created_at.htm
                ├── _retrieve_dt.htm
                ├── _retrieve_status.htm
                ├── _updated_at.htm
        ├── oedsapiuploadloghistory/
                ├── _auditlog_relation.htm
                ├── _completed_dt.htm
                ├── _created_at.htm
                ├── _livedata_relation.htm
                ├── _next_retry_dt.htm
                ├── _start_upload_dt.htm
                ├── _updated_at.htm
                ├── _updated_status.htm
        ├── oedsapiuploadloglive/
                ├── _auditlog_relation.htm
                ├── _completed_dt.htm
                ├── _created_at.htm
                ├── _livedata_relation.htm
                ├── _next_retry_dt.htm
                ├── _start_upload_dt.htm
                ├── _updated_at.htm
```

```
                ├── _updated_status.htm
        ├── oedsapiuploadloglivedata/
                ├── _rec_id.php
                ├── _rec_type.php
        ├── oedsauditlog/
                ├── _action.htm
        ├── oedsbatchapilog/
                ├── _created_at.htm
                ├── _updated_at.htm
        ├── oedssftpuploadlog/
                ├── _completed_dt.htm
                ├── _created_at.htm
                ├── _exporting_status.htm
                ├── _next_retry_dt.htm
                ├── _rec_end_dt.htm
                ├── _rec_start_dt.htm
                ├── _request_dt.htm
                ├── _start_upload_dt.htm
                ├── _type.htm
                ├── _updated_at.htm
                ├── _upload_status.htm
        ├── NmsSftpRetrievallog.php
        ├── OedsApiUploadlogHistory.php
        ├── OedsApiUploadlogLive.php
        ├── OedsBatchApiLog.php
        ├── OedsSftpUploadlog.php
├── models
        ├── nmssftpretrievallog/
                ├── _filepath.php
        ├── oedsapiuploadloglivedata/
                ├── _rec_id.php
                ├── _rec_type.php
        ├── oedsauditlog/
                ├── apicolumns.yaml
                ├── apifields.yaml
        ├── oedssftpuploadlog/
                ├── _filepath.php
        ├── NmsAuditlog.php
        ├── NmsSftpRetrievallog.php
        ├── OedsApiUploadlogHistory.php
        ├── OedsApiUploadlogHistoryData.php
        ├── OedsApiUploadlogLive.php
        ├── OedsApiUploadlogLiveData.php
        ├── OedsAuditlog.php
        ├── OedsBatchApilog.php
        ├── OedsBatchCheckstatuslog.php
        ├── OedsBatchVerifyfileslog.php
        ├── OedsSftpUploadlog.php
```

4.16.2 Module Components

Folder	File	Definition

controllers/nmssftpretrievallog	_completed_dt.htm _created_at.htm _retrieve_dt.htm _retrieve_status.htm _updated_at.htm	Shows the value for different fields in the “NMS Logs - Retrieval Log (SFTP)” list table
controllers/oedsapiuploadloghistory	_completed_dt.htm _created_at.htm _next_retry_dt.htm _start_upload_dt.htm _updated_at.htm _upload_status.htm	Shows the value for different fields in the “Periodic Upload Log (API)” list table
	_auditlog_relation.htm	Loads the audit log relation and shows in the record edit and preview page
	_livedata_relation.htm	Loads the “Record data list” relation and shows in the record edit and preview page. It includes the data records which are included in this upload
controllers/oedsapiuploadloglive	_completed_dt.htm _created_at.htm _next_retry_dt.htm _start_upload_dt.htm _updated_at.htm _upload_status.htm	Shows the value for different fields in the “Historical Upload Log (API)” list table
	_auditlog_relation.htm	Loads the audit log relation and shows in the record edit and preview page
	_livedata_relation.htm	Loads the “Record data list” relation and shows in the record edit and preview page. It includes the data records which are included in this upload
controllers/oedsapiuploadlogliveda ta	_rec_id.php	Shows the json format which is submitted to the OEDS system in the list of “Record data list” relation
	_rec_type.php	Shows Type of record in the list of “Record data list” relation
controllers/oedsauditlog	_action.htm	Shows Action type of record in the list of “Audit Log list” relation
controllers/oedsbatchapilog	_created_at.htm _updated_at.htm	Shows the value for different fields in the “Batch API Log” list table
controllers/oedssftpuploadlog	_completed_dt.htm _created_at.htm _exporting_status.htm _next_retry_dt.htm _rec_end_dt.htm _rec_start_dt.htm _request_dt.htm _start_upload_dt.htm _type.htm	Shows the value for different fields in the “Batch Upload Log (SFTP)” list table

	_updated_at.htm _upload_status.htm	
controllers	NmsSftpRetrievallog.php	Controls the list, preview, export function of "NMS Logs - Retrieval Log (SFTP)"
	OedsApiUploadlogHistory.php	Controls the list, preview, export function of "OEDS Logs - Historical Upload Log (API)"
	OedsApiUploadlogLive.php	Controls the list, preview, export function of "OEDS Logs - Periodic Upload Log (API)"
	OedsBatchApiLog.php	Controls the list, preview, export function of "OEDS Logs - Batch Upload Log (SFTP)"
	OedsSftpUploadlog.php	Controls the list, preview, export, create function of "OEDS Logs - Batch API Log"
models/nmssftpretrievallog	_filepath.php	Shows the downloaded log fie
models/oedsapiuploadloglivedata	_rec_id.php _rec_type.php	Shows the json and record type for the detail of record in "Record data list" relation
models/oedsauditlog	apicolumns.yaml apifields.yaml	Controls the list column and fields for "Audit Log Relation"
models/oedssftpuploadlog	_filepath.php	Shows the uploaded log fie
models	NmsAuditlog.php	Works with dcs_nms_audit_log table.
	NmsSftpRetrievallog.php	Works with dcs_logpage_nmssftpretrievallog table.
	OedsApiUploadlogHistory.php	Works with dcs_logpage_oedsapiuploadloghistory table and configured the relation tables
	OedsApiUploadlogHistoryData.php	Works with dcs_logpage_oedsapiuploadloghistory_data table.
	OedsApiUploadlogLive.php	Works with dcs_logpage_oedsapiuploadloglive table and configured the relation tables
	OedsApiUploadlogLiveData.php	Works with dcs_logpage_oedsapiuploadloglive_data table.
	OedsAuditlog.php	Works with dcs_oeds_audit_log table.
	OedsBatchApilog.php	Works with dcs_oeds_batch_api_log table.
	OedsBatchCheckstatuslog.php	Works with dcs_oeds_batch_checkstatus_log table.
	OedsBatchVerifyfileslog.php	Works with dcs_oeds_batch_verifyfiles_log table.

	OedsSftpUploadlog.php	Works with dcs_logpage_oedssftpuploadlog table.

4.16.3 Module Functions

4.16.3.1 Show “NMS - Retrieval Log (SFTP)” section

Shows the “NMS Logs -> Retrieval Log (SFTP)” section on CMS. Supports filtering of the list and export the records to CSV file. In addition, it supports downloading the raw file. Each record includes the following fields :

Field Name	Definition	Support filter independently
ID	Record ID	No
Filename	The name of retrieved file	No
#Records	No of records retrieved in this file	No
Start Retrieval time	Start download time	Filter by date range
Complete time	Complete processing time	Filter by date range
Retrieved status	Status of the retrieval (refer 4.15.4.1)	Filter by checklist
Created at	Created time	Filter by date range
Updated at	Updated time	Filter by date range

4.16.3.2 Show “OEDS - Periodic Upload Log (API)” section

Shows the “OEDS Logs -> Periodic Upload Log (API)” section on CMS. Supports filtering of the list and export the records to CSV file. Each record includes the following fields :

Field Name	Definition	Support filter independently
ID	Record ID	No
#Records	No of records retrieved in this file	No
Start Upload time	Start upload time	Filter by date range
Complete time	Complete processing time	Filter by date range
#Attempt	Number of submit attempts	No
Next Retry time	Next retry time	No
Upload status	Status of upload action (refer 4.15.4.2)	Filter by checklist
Created at	Created time	Filter by date range
Updated at	Updated time	Filter by date range

4.16.3.3 Show “OEDS - Historical Upload Log (API)” section

Shows the “OEDS Logs -> Historical Upload Log (API)” section on CMS. Supports filtering of the list and export the records to CSV file. The record fields are the same as Section 4.15.3.2.

4.16.3.4 Show “OEDS - Batch Upload Log (SFTP)” section

Shows the “OEDS Logs -> Batch Upload Log (SFTP)” section on CMS. Supports filtering of the list and export the records to CSV file. Each record includes the following fields :

Field Name	Definition	Support filter independently
ID	Record Id	No
Filename	Filename	No
#Records	No of Record	No
Request Data type	Requested type of data (refer 4.15.4.3)	Filter by checklist
Record - start period	Requested start period	Filter by date range
Record - end period	Requested end period	Filter by date range
Request time	Request time	Filter by date range
Start Upload time	Start uploading file time	Filter by date range
Complete time	Complete time	Filter by date range
File exporting status	File export status (refer 4.15.4.4)	Filter by checklist
#Attempt	No of attempt	No
Next Retry time	Next retry time	Filter by date range
Upload status	Upload status (refer 4.15.4.5)	Filter by checklist
Created at	Created time	Filter by date range
Updated at	Updated time	Filter by date range

4.16.3.5 Show “OEDS - Batch API Log” record section

Shows the “OEDS Logs -> Batch API Log” section on CMS. Supports filtering of the list and export the records to CSV file. In addition, it supports downloading the raw file. Each record includes the following fields :

Field Name	Definition	Support filter independently
ID	Record Id	No
IP address	IP address	No
Parameters	Received parameters	No
Response	API Response	No
Created at	Created time	Filter by date range
Updated at	Updated time	Filter by date range

4.16.4 Implementation Details

4.16.4.1 “Retrieved Status” Definition

The retrieval status of the records

Value	Meaning
1	Pulled
2	Start Parsing
3	Parsing successfully
4	Parsing failed
5	Pull failed

4.16.4.2 “Upload Status” Definition

The upload status of the records

Value	Meaning
1	Periodic Post Start
2	Periodic Post Successfully
3	Periodic Post Failed

4.16.4.3 Requested type of files

The type of files that allow to be requested for historical upload

Value	Meaning
2	Alarm

4	Heartbeat

4.16.4.4 OEDS Batch upload - File exporting status

File exporting status for OEDS Batch upload request

Value	Meaning
2	Pending for exporting
3	Processing the export process
4	Export completed

4.16.4.5 Upload Status

Status for showing the progress of batch request

Value	Meaning
4	Batch request received
5	File preparation start
6	File preparation completed
7	File preparation failed
8	Upload start
9	Upload (Retry) start
10	Upload end successfully
11	Upload end failed
12	Verify files have been processed by OEDS successfully